

# EXERCISE 7: Get Started with Flower Framework

1. In the basic Flower setup described in the tutorial, what are the two main applications that the user must define? What are their respective roles?

Flower requires defining two applications:

- **ClientApp**  
This runs on every client device and specifies what each client does locally. It contains the training and evaluation functions (`@app.train`, `@app.evaluate`) that describe how a client trains on its own data, updates the model, and evaluates locally.
- **ServerApp**  
This runs on the central server and controls the overall federated learning process. It defines the strategy (FedAvg), configures each round, and aggregates the updates coming from clients.

2. How do clients and servers communicate and share parameters in Flower? Describe the object they use and the purpose of each specific field.

In Flower, the server and clients communicate by sending and receiving **Message** objects. A Message carries a RecordDict as its main payload. The RecordDict is like a Python dictionary that can contain multiple records of different types. There are three main types of records:

- **ArrayRecord**: Contains model parameters as a dictionary of NumPy arrays
- **MetricRecord**: Contains training or evaluation metrics as a dictionary of integers, floats, lists of integers, or lists of floats.
- **ConfigRecord**: Contains configuration parameters as a dictionary of integers, floats, strings, booleans, or bytes. Lists of these types are also supported.

3. How can a user define how a client should perform training? Is there any constraint on the name of the training function?

We can define how the ClientApp performs training by wrapping a function with the `@app.train()` decorator. The function name does not matter. It can be anything.

4. What is the difference between implementing an `@app.evaluate` function on a server and on a client?

- **Client-side evaluate**

Runs on each client and evaluates the global model on local client data. The server collects these results and aggregates them.

- **Server-side evaluate**

Runs only on the server, usually using a central validation dataset. It evaluates the global model directly without involving clients.

5. What is the purpose of the `Context` object? How can we modify the simulation parameters?

**Context:** The context object that contains information about the node executing the ClientApp and about the current run.

Through the context you can retrieve the config settings defined in the `pyproject.toml` of your app. The context can be used to persist the state of the client across multiple calls to train or evaluate. In Flower, ClientApps are ephemeral objects that get instantiated for the execution of one Message and destroyed when a reply is communicated back to the server.

## EXERCISE 8: Tackle device Heterogeneity

1. What is system heterogeneity and why it is a problem in Federated Learning?

System heterogeneity refers to the significant differences across client devices in terms of computing power, memory, network bandwidth, battery, and availability. In federated learning, this becomes a problem because slow or resource-constrained devices cannot train large models or keep up with round deadlines, leading to “stragglers.” This causes unstable training, biased updates from only powerful devices, and reduced fairness and accuracy in the global model.

## 2. What is Ordered Dropout?

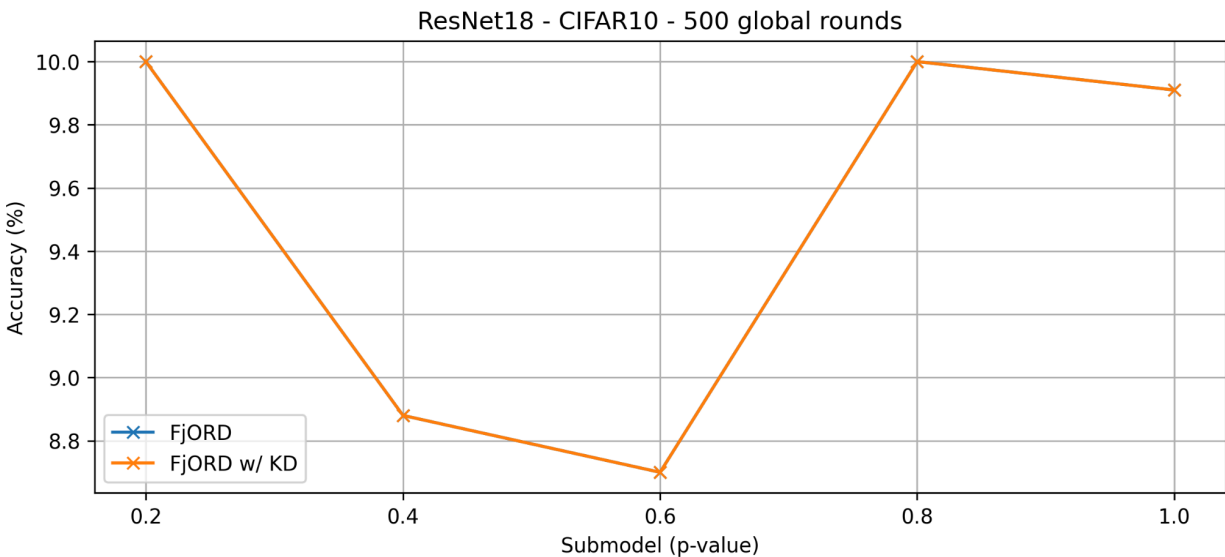
Ordered Dropout is a technique introduced in FjORD that allows each client to train only a prefix (smaller subset) of the model layers based on its capacity. Instead of dropping arbitrary neurons, layers or channels are ordered from most essential to least essential, and weaker devices train only the earlier, smaller portion of the model. This ensures that every client contributes consistently while avoiding overloading low-resource devices.

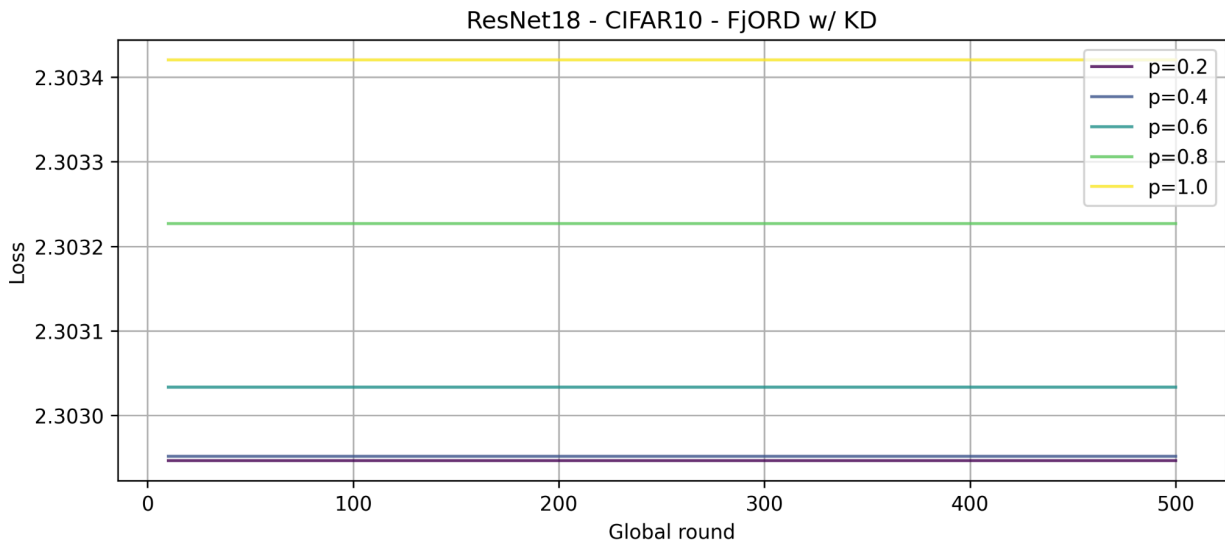
## 3. How does the aggregation rule account for device heterogeneity?

FjORD modifies the aggregation rule so that each client's update is aggregated only for the part of the model it actually trained. Higher-capacity devices update the full model, while lower-capacity devices update only the shared prefix they trained. The server aligns these partial updates by combining prefixes proportionally and using full updates only where they exist, ensuring fair contribution without penalizing weaker devices.

## FjORD baseline

### Analysis: Plots





**1. Which one of the two implementation (with and without knowledge distillation) works better? Why?**

The KD implementation is **better** in terms of global and submodel accuracy, especially when clients can train larger prefixes. It helps align knowledge across model widths, improving fairness and performance. Hence, simply stated self-distillation is the reason why KD implementation is better.

**2. How do different values of  $p$  impact the model's accuracy? Motivate your answer.**

In Fjord, larger  $p$  (wider submodels) yield higher accuracy, while very small  $p$  limits learning. Knowledge distillation boosts performance, especially for intermediate-to-large  $p$ , and ordered dropout ensures all clients contribute. Gains flatten near full model width, so optimal  $p$  balances accuracy and heterogeneity.

This trend can also be seen in the plot above.